

Governed AI Database Copilot

A multi-agent, RAG-grounded, MCP-powered assistant that lets anyone talk to their own database safely

1. The Real-World Problem

Most people who need data from a database aren't SQL experts. A store manager wants to know "which products are running low this month," a support lead wants "customers who complained twice," but both have to wait on an engineer to write and run the right query. That's slow, and it doesn't scale as a team grows.

AI chatbots can write SQL now — but handing an AI direct, unsupervised access to a real database is dangerous. A single wrong or misunderstood prompt ("delete inactive users") could wipe out real records with no undo. Most "chat with your database" tools in the market today focus on convenience and skip this safety problem almost entirely.

Real-world analogy: it's like giving a new intern the master key to the building on day one, versus giving them a badge that only opens the rooms they need, and requires a manager's sign-off before they touch anything expensive. Most AI database tools hand out the master key. This project builds the badge system.

2. The Solution, in Plain Terms

An application where a user connects their own database (their own login credentials, their own schema), then simply types what they want in plain English. Behind the scenes, a team of specialized AI agents works together to understand the request, look up the real table and column names, write the correct query, and — critically — check whether the action is safe before doing anything that changes data. Any risky action is shown to the user in plain language first, and only runs after they confirm it.

Think of it less like a single chatbot and more like a small office: a receptionist who takes the request, a researcher who looks up the relevant files, a clerk who drafts the paperwork, a supervisor who reviews anything risky before it's signed, and an assistant who actually files it and reports back.

3. How It Works — The Agent Team

Planner agent — Reads the user's prompt and figures out: is this a read, a write, or something ambiguous that needs clarifying? Breaks multi-step requests into smaller steps.

Clarifier agent — Checks whether the request has more than one reasonable interpretation. If it does, it does not guess — it stops and asks the user to choose before anything is generated.

Retriever agent (RAG) — Searches a per-database index of table names, columns, and business glossary terms, and pulls back only the pieces relevant to this specific question. Refreshes its index at the start of each session so it never works from a stale schema.

SQL Generator agent — Writes the actual SQL query, using the real schema details the Retriever just handed it — not guessed column names. Output is forced into a validated, structured format (not loose free text), so a malformed query is caught before it ever reaches the database.

Safety Critic agent — A separate check — reviews the generated query for risk (delete, update, mass changes, sensitive tables) and assigns a risk level.

Executor (via MCP) — The only component with real database access. Runs safe reads immediately; for risky writes, it waits for the user's explicit confirmation, then runs inside a transaction it can undo. If a query errors (bad column, syntax issue), the error is fed back to the SQL Generator for exactly one self-correction retry before failing gracefully with a clear explanation.

Explainer agent — Turns raw query results, or a pending risky action, back into plain language the user can understand and act on.

Splitting this into separate agents matters for one key reason: the agent that writes the SQL should not also be the one that decides if it's safe. That's the same reason a bank doesn't let the teller who processes a wire transfer also approve it — a second, independent check catches mistakes the first one would miss.

Handling Ambiguous Requests — A Worked Example

Some prompts sound simple but actually hide more than one valid meaning. Example: the user asks "select the best employee this month." "Best" could reasonably mean most hours worked, or most tasks completed — and picking one silently would give a confident-sounding but possibly wrong answer.

In this system, the Clarifier agent catches this before any SQL is written. Instead of guessing, it responds: "By 'best employee,' do you mean the most hours worked, or the most tasks completed this month?" and waits for the user to pick. Only once the metric is confirmed does the Retriever and SQL Generator run. This is the same principle as the write-confirmation flow, applied one step earlier: when the system isn't sure what's being asked, it asks — it never quietly assumes.

Real-world analogy: a good waiter doesn't guess when you order "the usual" on your first visit — they ask what you mean instead of bringing out the wrong dish.

4. Why RAG (Retrieval-Augmented Generation) Is Needed

An AI model doesn't automatically know what tables and columns exist in a stranger's database — and every user connects a different one. Without help, it would guess column names and get them wrong. RAG solves this by acting like a librarian: instead of handing the AI the entire library (every table in the database) for every single question, it quickly finds the few relevant "books" (tables, columns, glossary terms) and hands over just those.

Step A — Ingestion (once, when a database is connected):

- The app reads the real schema: table names, column names, relationships.
- An AI pass auto-drafts a plain-language glossary (e.g. "churned customer" maps to a specific column and condition), which the user can correct.
- All of this is broken into small chunks and converted into vectors (number representations of meaning), stored in a private index for that one database connection only.

Step B — Retrieval (every time the user asks something):

- The user's question is converted into a vector the same way.
- The system searches the index for the closest-matching schema pieces and glossary terms.
- Only those relevant pieces — not the whole database — are added to the AI's prompt before it writes the SQL.

Real-world example: a hospital receptionist doesn't memorize every patient's full file. When a patient calls, they pull only that patient's folder from the records room. RAG is the records room lookup step — it finds the right folder before anyone tries to answer the question.

5. Safety & Confirmation Flow

This is the feature that separates the project from a basic "AI writes SQL" demo, and the part most worth highlighting in an interview.

- Read-only actions (SELECT) run immediately — there's nothing to lose.
- Any write (UPDATE, DELETE, INSERT at scale) is first shown as a preview: the generated SQL, plus a plain-language summary like "this will update 14 rows in orders, delete 2 in refunds."
- The user must explicitly confirm before it runs.
- Every write happens inside a database transaction, so it can be rolled back within a short window if something looks wrong.
- Every prompt, generated query, and outcome is written to an audit log the user can review later.
- The database connection itself defaults to read-only; write access is a separate, explicit opt-in per connection.

Real-world analogy: online banking apps let you check your balance instantly, but a large transfer always shows you a confirmation screen with the amount and recipient before it goes through. This project applies that same pattern to database operations.

6. Suggested Tech Stack

Layer	Choice	Why
Agent orchestration	LangGraph	Explicit state machine — fits a multi-agent flow with a clarification and confirmation pause
Database access	MCP server (Python SDK)	Keeps raw DB credentials away from the agents — only scoped tool calls are exposed
Structured output	Instructor / Pydantic	Forces the SQL Generator's output into a validated schema instead of loose free text
Vector store	Qdrant	Per-connection namespaces for schema/glossary RAG
Database	PostgreSQL / MySQL	Widely used, easy to demo with a seeded sample dataset
LLM	Claude / GPT via API	Separate calls for the SQL Generator and the Safety Critic roles
Frontend	Next.js	Chat UI + confirmation modal + audit-log dashboard

7. 20-Day Build Plan

Days	Focus	Tasks
------	-------	-------

1–5	Foundations	Connection UI + credential vault, MCP server with scoped DB tools, schema introspection, auto-glossary draft
6–10	Core agent flow	Planner + Clarifier + SQL Generator agents with structured output, per-tenant RAG indexing in Qdrant, LangGraph state flow for reads
11–15	Safety layer	Safety Critic agent, transaction-wrapped writes, rollback logic, confirmation modal UI, plain-language diff preview, permission toggles
16–18	Trust & proof	Self-correction retry loop, audit log storage + dashboard, eval harness (correctness, safety, and ambiguity-handling scoring)
19–20	Polish & demo	Bug fixes, edge cases, README, architecture diagram, demo script

8. Real-World Walkthrough — One Full Prompt

A user has connected their store's database and types: *"Which customers haven't ordered in the last 90 days?"*

1. Planner reads the prompt: this is a read, no write involved — low risk, no confirmation needed.
2. Retriever searches the vector store and finds the customers and orders tables, plus the glossary entry defining "inactive customer."
3. SQL Generator writes a query joining those two tables with the correct date condition, using the real column names it was just handed.
4. Safety Critic reviews it, confirms it's a read-only SELECT, and clears it to run immediately.
5. Executor runs the query through the MCP tool using the connection's read-only credentials.
6. Explainer turns the raw rows into a short plain-language summary and a results table for the user.

Now suppose the same user follows up with: *"Delete all customers who haven't ordered in 2 years."* The flow is identical through the Retriever and SQL Generator, but the Safety Critic flags this as high risk (a DELETE affecting many rows). Instead of running it, the Explainer shows: "This will permanently delete 128 customer records. Confirm?" — and nothing happens until the user taps confirm.

One more case: the user types *"select the best employee this month."* The Planner passes this to the Clarifier before any schema lookup happens. The Clarifier detects two valid readings of "best" — hours worked vs. tasks completed — and responds with a direct question instead of a guess: "Do you mean most hours worked, or most tasks completed?" Only after the user answers does the Retriever and SQL Generator run, against the metric the user actually meant.

Benchmark — Sample Eval Table

Modeled on how production text-to-SQL systems report accuracy: run a fixed test set of real questions against the agent and track the outcome, including how often it correctly asks for clarification instead of guessing.

Metric	Target
Correct SQL on first try	≥ 75%
Correct after 1 self-correction retry	≥ 90%
Ambiguous questions correctly flagged for clarification	100%
Destructive queries correctly intercepted for confirmation	100%

9. Why This Stands Out in an Interview

- It's not just prompt engineering — it demonstrates real systems thinking: multi-agent design, tenant isolation, least-privilege access, and auditability.
- The built-in evaluation harness shows a skill most student projects skip entirely: measuring whether an AI system is actually correct and safe, not just demoing it once.

- MCP is a current, recognizable piece of the modern AI stack — using it cleanly separates "agent reasoning" from "real system access," a distinction interviewers care about.
- The safety-first framing (dry-run previews, confirmation, rollback, audit trail) is the difference between a toy demo and something that could plausibly be trusted in production.
- Asking for clarification on ambiguous requests, instead of silently guessing, is a detail almost every text-to-SQL project skips — and it's an easy one to demo live and explain under questioning.